

OOPS one shot

→ objects

definition + example.

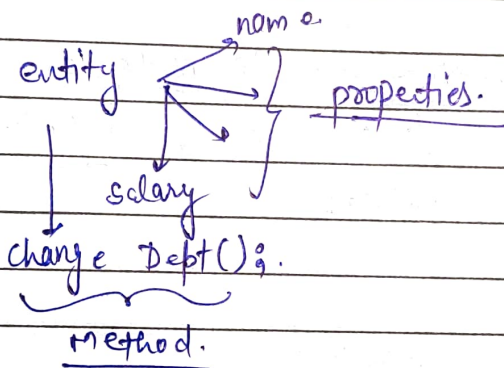
↓

vector

, string, stack } → STL

Classes & objects -

- objects are entities in the real world.
- class is like a blueprint of these entities.
group of object.



class → BP

obj 1	T ₁	50/w
obj 2	T ₂	
obj 3	T ₃	

Syntax →

class Teacher {
 properties
 method.

};

this is

important that class
is not a function!

class Teacher {

⊗ → // properties / attributes

string name;

string dept;

float salary;

// Methods / member functions

⊗ void changeDept (string newDept)

{ Dept = newDept
}

};

Objects -

Teacher T₁;

Teacher T₂;

T₁.name = "Shreehar"

T₁.Dept ~~T₁.subject~~ = "CSE"

T₁.salary = "25000"

We can't access any thing like
so we need access modifier

Access Modifier ⇒

private - data and methods accessible inside class.

public - data & methods accessible to everyone

protected - data & methods accessible inside class & to its derived class. (use in inheritance)

public जिससे सब को access कर सकते हैं

private: -

public: -

class Teacher {

private:

double salary;

public:

string name;

string dept;

setter & getter function. → to get the private value

↳ to set the private value

```
void setSalary(double s) {
    s = salary;
}

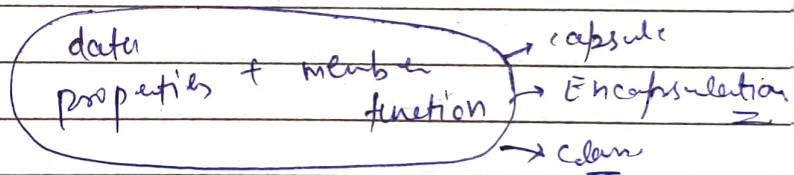
double getSalary() {
    return salary;
}
```

oops

- ① Encapsulation
- ② Abstraction
- ③ Inheritance
- ④ Polymorphism

*** Encapsulation - (capsule या गोदम)

Encapsulation is wrapping up of data & member functions in a single unit called class.



class के अंदर data और function लिखा है और Encapsulation है।

Helps in data hiding - private:

Construction

→ Constructor - special method invoked automatically at time of object creation. Used for initialization.
→ Same name as class.

① → Constructor doesn't have a return type.

→ Only called once automatically, at object creation.

→ memory allocation happens when const. called.

constructor

- Non parameterized constructor
- Parameterized constructor
- copy constructor

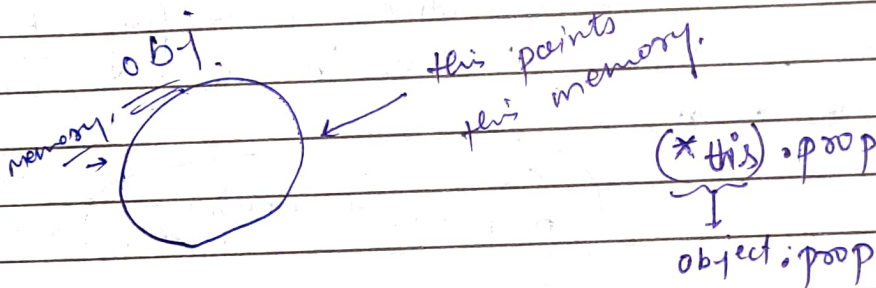
type diff: constructor overloading. } Polymorphism

Multiple constructor with different pairing values.

this pointer -

this is a special pointer in C++ that points to the current object.

this → prop is same as $*(this).prop$



Copy constructor -

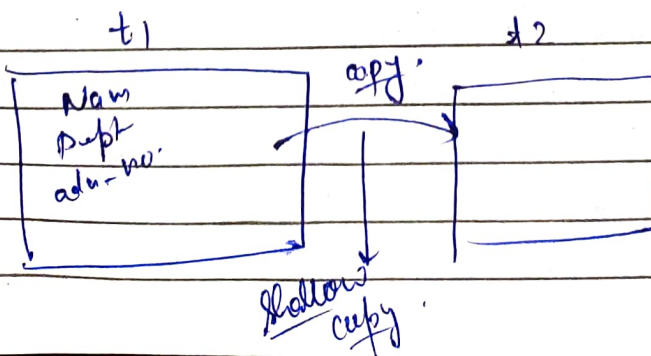
special constructor (default) used to copy properties of one object into another

Teacher t1 (———→)

↳ copy ↳ copy to gaur

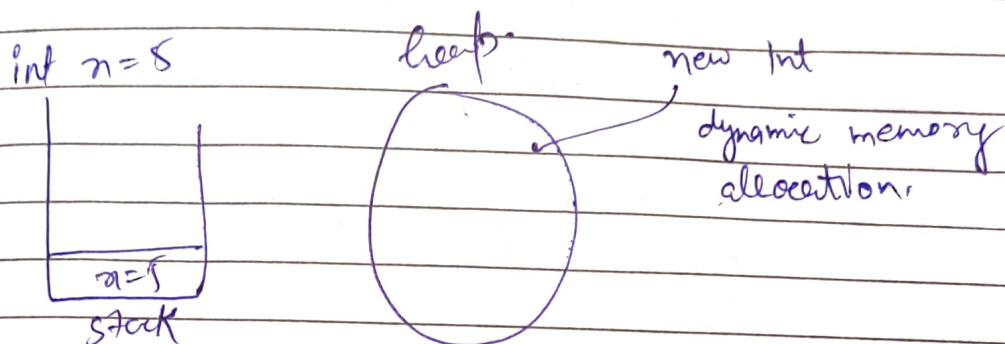
Teacher t2(t1);

↳ const Teacher d.

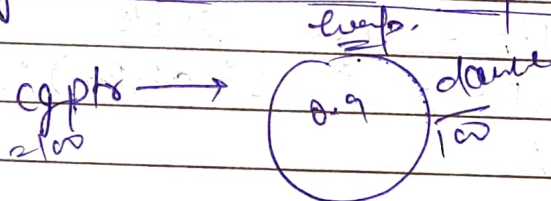


Shallow & Deep copy

Shallow - A shallow copy of an object copies all of the member values from one object to another.



Deep copy - A deep copy, on other hand, not only copies the member value but also makes copies of any dynamically allocated memory that the member point to.



cgptrs = new double;

* cgptrs = cgs;

s1.getinfo() → cgs = 8.9

Student s2(s1);

* (s2.cgptrs) = 9.2

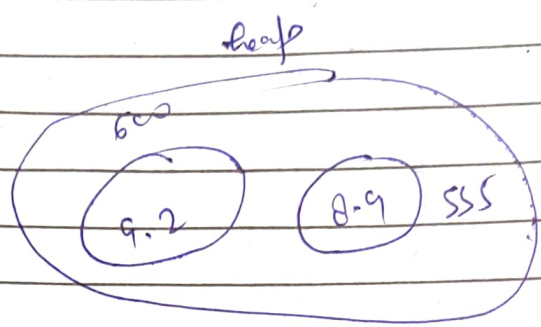
s1.getinfo() → cgs = 8.9

s2 change the value of
which is s1's value copy
change the value / Shallow copy as it is

Shallow copy same address copy करता है इसीलिए बाद वाले का change करने से पहले वाला भी change हो जाता है।

so we need deep copy -

→ ऐसा construct स्वयं लिखें



copy constructor →

deep copy

Student (Student &obj){

 this->name = obj.name;

 cgpaPtr = new double

 *(cgpaPtr) = *(obj.cgpaPtr);

}

Destructor ⇒ opp of constructor

deallocator

allocator

हमें dynamic allocate memory को self से delete करना पड़ता है।

new → delete

delete ptr

delete करता है
us memory को जिसे
ptr point कर रहा होता
है।

const &ptr &.
ptr जिसे भी exist
करता है।

→ code reusability.

Date: _____

Page: _____

Inheritance.

when properties & member function of base class are passed on to the derived class.

parents

↓
child

inheritance. (विरासत)

(int x) class A (Parent / Base class)

(inherited)

↓

↓ inherit.

(int x) class B (Child / Derived class)

Constructors → पहले call होगी और फिर पहले parent call होगा और फिर child call होगा।

~~Records~~

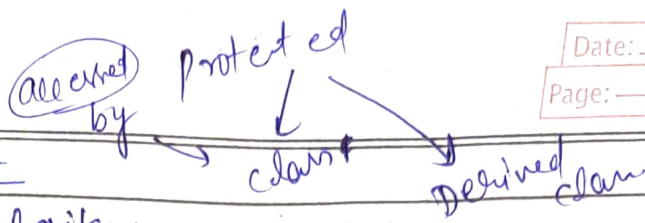
Destructors → पहले child call और फिर parent .clear होगा।

→ calling parametrized constructor of Person.

student (string name, int age, string rollno) : Person
(name, age) {

this → rollno = rollno.

}



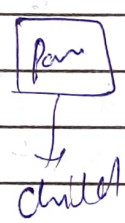
Inheritance-

Mode of Inheritance-

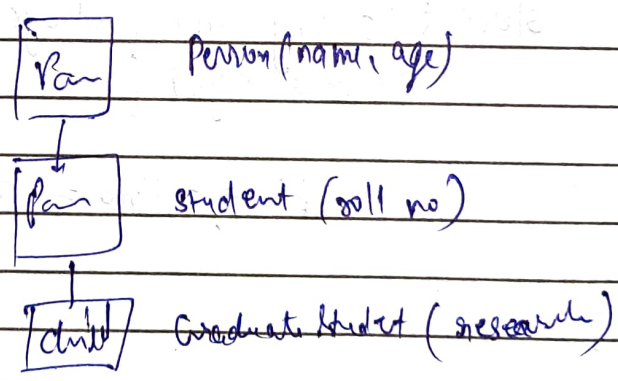
	Derived Class	Derived	Derived
Base/Parent class	Private	Protected	Public
→ Private	Not Inherited	Not Inherited	Not Inherited
Protected	Private	Protected	Protected
Public	Private	Protected	Public

Types of Inheritance-

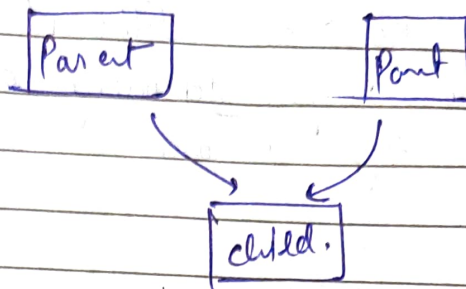
① Single Inheritance-



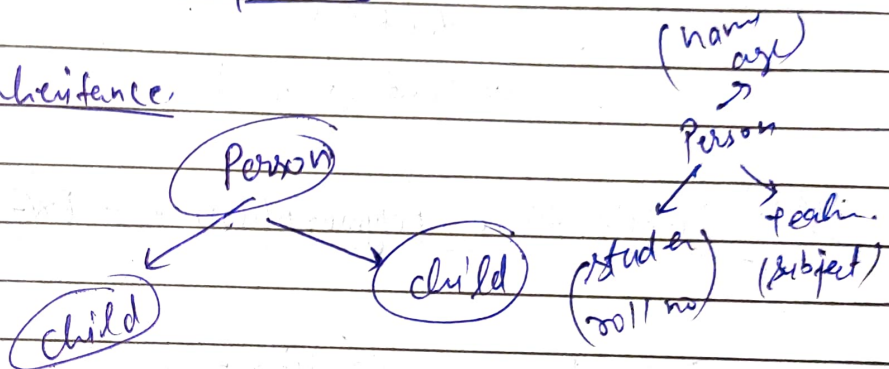
② Multi-level Inheritance-



Multiple Inheritance

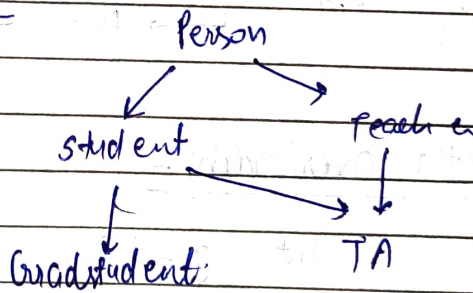


Hierarchical Inheritance



class TA : public Teacher, public student;

Hybrid Inheritance



example → constructor overloading.
multiple constructor

Date: _____

Page: _____

Polymorphism-

Polymorphism is the ability of objects to take on different forms or behave in different ways depending on the context in which they are used.

like

→ student()

→ student(name)

statically-

Compile time polymorphism → Ex: constructor overloading

② Function overloading-

class {
 name should be same {
 fun() → parameters
 fun() → param
 }
}

different

③ operator overloading-

int y = 10
int x = y } x = 10

int a = 5

int b = 10

cout << a + b;

```
class complex {  
    int real, img;  
    complex (int real, int img) {  
        this->real = real;  
        this->img = img;  
    }  
}
```

```
int main {  
    complex c1 (3, 2);  
    complex c2 (4, 6);  
    complex c3 = c1 + c2;  
    c3->display();  
}
```

Operators that cannot be overload →

- ① Scope Resolution Operator '::'
- ② member Access Operator '.' (dot)
- ③ Conditional (Ternary) Operator '? :'
- ④ sizeof operator 'sizeof'
- ⑤ typeid operator 'typeid'
- ⑥ ~~Assignment operator~~

Runtime Polymorphism

- Function Overriding -

Parent & child both contain the same function with different implementation. The parent class function is said to be overridden.

Overload. → same class

Overriding → Inheritance.

virtual function-

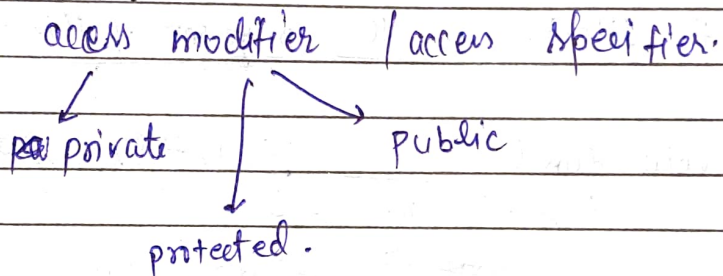
A virtual function is a member function that you expect to be redefined in derived classes.

- Virtual functions are dynamic in nature.
- Defined by the keyword "virtual" inside a base class and are always declared with a base class and overridden in the child class.
- A virtual function is called during runtime.

Abstraction-

sensitive

Hiding all unnecessary details and showing only the important parts.

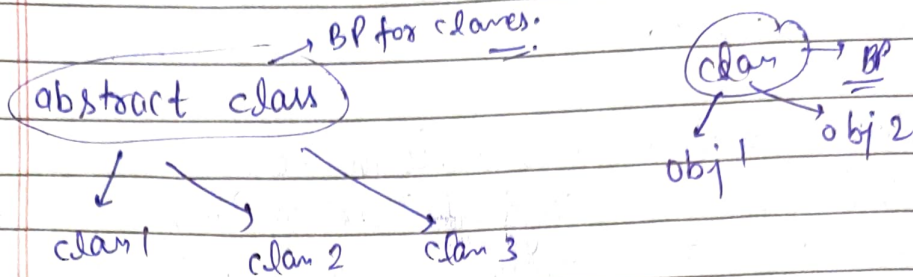
using Abstract classes-

→ Abstract classes are used to provide a base class from which other classes can be derived.

→ They cannot be instantiated and are meant to be inherited.

obj

- Abstract classes are typically used to define an ~~int~~ interface for derived classes



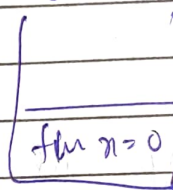
Static keyword -

• Static Variables

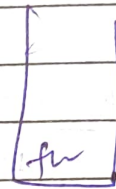
→ Variable declared as static

in a function are created & initialised once for the lifetime of the programme. // in function.

with static



with static



```
void fun ()
```

```
{ static int n = 0;
```

```
  cout << n << endl;
```

```
  n++;
```

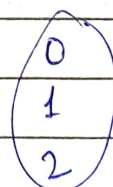
```
}
```

```
int main () {
```

```
  fun(); fun(); fun();
```

```
}
```

Output is

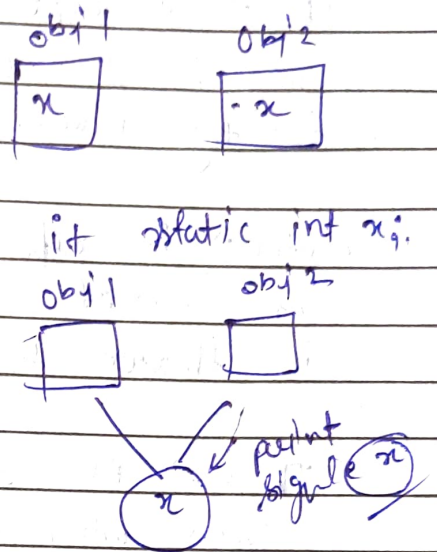


→ static variables in a class are created & initialised once. They are shared by all the objects of the class. // in class

```
class A {
public:
    int x;
    void incX() {
        x++;
    }
};
```

```
int main () {
```

```
    A obj;    obj.x = 0
    cout << obj.x << endl;
    obj.incX();
    cout << obj.x << endl;
    return 0;
}
```



• Static objects → life time of the program.

Exception Handling-

- **TRY** → It represents a block of code that may throw an exception placed inside the try block.
- **Catch** → It represents a block of code that is executed when a particular exception is thrown from the try block.
- **Throw** → An exception in C++ can be thrown using the throw keyword.